

1 Метод Гаусса решения СЛАУ

Задача: решить СЛАУ $Ax = b$ методом Гаусса с выбором главного элемента в столбце, где

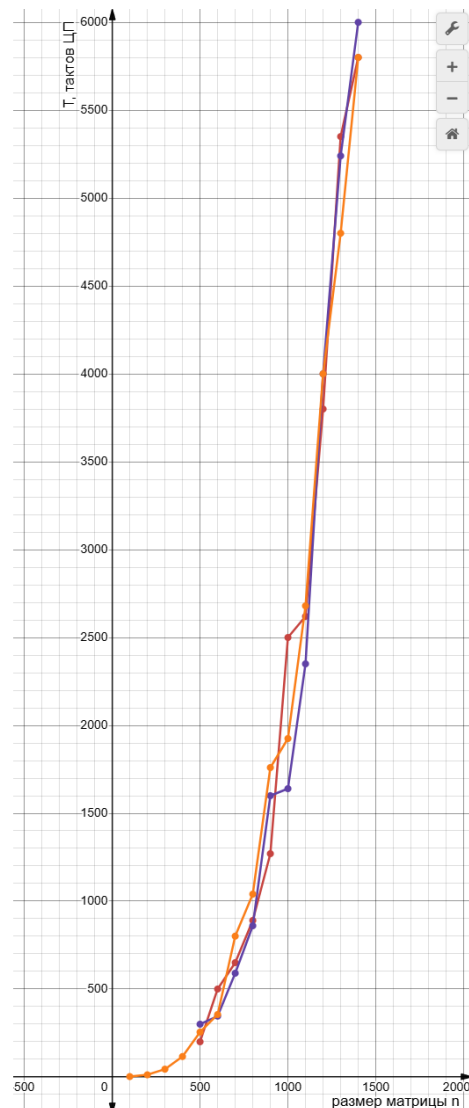
$$A = \begin{pmatrix} 3 & 1 & -1 & 0 & 3 & 2 \\ 1 & -2 & 0.5 & 4 & -1 & 3 \\ 0 & 3 & 1 & -2 & 2 & 1 \\ 1 & 0 & -3 & 2 & 1 & -1 \\ 2 & -1 & 1 & 3 & -2 & 0.5 \\ -1 & 2 & 0 & 1 & 4 & -3 \end{pmatrix} \quad b = \begin{pmatrix} 4 \\ 2 \\ 5 \\ 3 \\ 1 \\ 6 \end{pmatrix}.$$

С помощью написанной программы получено решение, выполнена проверка корректности решения $Ax - b = 0$, построен график зависимости времени выполнения от размера матрицы A .

$$x = \begin{pmatrix} -0.162611 & 2.253364 & -0.166619 & 1.402806 & 0.356427 & 0.499284 \end{pmatrix}^T,$$

```
> .\Gauss_main_el.exe
Введите 0 для генерации матрицы, 1 для чтения из файла: 1

=====
Матрица A\b:
3.00 1.00 -1.00 0.00 3.00 2.00 4.00
1.00 -2.00 0.50 4.00 -1.00 3.00 2.00
0.00 3.00 1.00 -2.00 2.00 1.00 5.00
1.00 0.00 -3.00 2.00 1.00 -1.00 3.00
2.00 -1.00 1.00 3.00 -2.00 0.50 1.00
-1.00 2.00 0.00 1.00 4.00 -3.00 6.00
Матрица после прямого хода (верхнетреугольная):
3.000 1.000 -1.000 0.000 3.000 2.000 4.000
0.000 3.000 1.000 -2.000 2.000 1.000 5.000
0.000 0.000 -2.556 1.778 0.222 -1.556 2.222
0.000 0.000 0.000 3.565 -0.304 2.130 5.957
0.000 0.000 0.000 0.000 3.500 -3.500 -0.500
0.000 0.000 0.000 0.000 0.000 -6.085 -3.038
Время выполнения: 0.001000 секунд
Процессорные такты: 1
Решение системы:
X[0] = -0.162611
X[1] = 2.253364
X[2] = -0.166619
X[3] = 1.402806
X[4] = 0.356427
X[5] = 0.499284
=====
check[0] = 0.000000
check[1] = 0.000000
check[2] = -0.000000
check[3] = 0.000000
check[4] = -0.000000
check[5] = -0.000000
```



Программная реализация на языке C:

```
1  #include <stdio.h>
2  #include <locale.h>
3  #include <stdlib.h>
4  #include <windows.h>
5  #include <math.h>
6  #include <time.h>
7
8
9  double* gauss(double **A, int n) {
10     // Прямой ход с выбором главного элемента по столбцу
11     for (int k = 0; k < n - 1; k++) {
12         // 1. Поиск главного элемента в столбце k
13         int max_row = k;
14         double max_val = fabs(A[k][k]);
15         for (int i = k + 1; i < n; i++) {
16             if (fabs(A[i][k]) > max_val) {
17                 max_val = fabs(A[i][k]);
18                 max_row = i;
19             }
20         }
21         // Проверка на ноль
22         if (max_val < 1e-12) {
23             printf("Ошибка: столбец %d почти нулевой (главный элемент = %g)\n",
24                 k, max_val);
25             return NULL;
26         }
27         // 2. Перестановка строк
28         if (max_row != k) {
29             double *tmp = A[k];
30             A[k] = A[max_row];
31             A[max_row] = tmp;
32         }
33         // 3. Исключение под диагональю
34         for (int i = k + 1; i < n; i++) {
35             double factor = A[i][k] / A[k][k];
36             for (int j = k; j < n + 1; j++) {
37                 A[i][j] -= factor * A[k][j];
38             }
39         }
40     }
41
42     // Вывод матрицы после прямого хода
43     if (n < 10) {
44         printf("Матрица после прямого хода (верхнетреугольная):\n");
45         for (int i = 0; i < n; i++) {
46             for (int j = 0; j < n + 1; j++) {
47                 printf("%.3f ", A[i][j]);
48             }
49             printf("\n");
50         }
51     }
52 }
```

```

50     }
51 }
52
53 // Обратный ход с проверкой диагонали
54 double *X = (double *)malloc(n * sizeof(double));
55 for (int i = n - 1; i >= 0; i--) {
56     if (fabs(A[i][i]) < 1e-12) {
57         printf("Ошибка: нулевой диагональный элемент на шаге %d\n", i);
58         free(X);
59         return NULL;
60     }
61     X[i] = A[i][n];
62     for (int j = i + 1; j < n; j++) {
63         X[i] -= A[i][j] * X[j];
64     }
65     X[i] /= A[i][i];
66 }
67 return X;
68 }
69
70 double* matrix_x_vector_column(double **A, double *X, int n) {
71     double *B = (double *)malloc(n * sizeof(double));
72     for (int i = 0; i < n; i++) {
73         B[i] = 0.0;
74         for (int j = 0; j < n; j++) {
75             B[i] += A[i][j] * X[j];
76         }
77     }
78     return B;
79 }
80
81 int main() {
82     SetConsoleOutputCP(CP_UTF8);
83     setlocale(LC_NUMERIC, "C");
84
85     double **A;
86     double **A_for_check;
87     double *B;
88     double *X;
89     double *X_target;
90     int n;
91
92     int is_use_file = 0; // 0 - не использовать файл, 1 - использовать файл
93     printf("Введите 0 для генерации матрицы, 1 для чтения из файла: ");
94     scanf("%d", &is_use_file);
95
96     if (is_use_file == 1) { // ИСПОЛЬЗУЕТСЯ ФАЙЛ
97         FILE *fp = fopen("matrixA.txt", "r");
98         // Чтение размера матрицы
99         if (fscanf(fp, "%d", &n) != 1) {

```

```

100     printf("Ошибка чтения размера матрицы\n");
101     fclose(fp);
102     return 1;
103 }
104 // A --- Расширенная матрица коэффициентов (n x (n+1))
105 A = (double **)malloc(n * sizeof(double *));
106 for (int i = 0; i < n; i++) {
107     A[i] = (double *)malloc((n + 1) * sizeof(double));
108 }
109 // Чтение расширенной матрицы A|b из файла
110 for (int i = 0; i < n; i++) {
111     for (int j = 0; j < n + 1; j++) {
112         if (fscanf(fp, "%lf", &A[i][j]) != 1) {
113             printf("Ошибка чтения элемента матрицы A[%d][%d]\n",
114                 i, j);
115             fclose(fp);
116             return 1;
117         }
118     }
119 }
120 //копируем матрицу A для проверки
121 A_for_check = (double **)malloc(n * sizeof(double *));
122 for (int i = 0; i < n; i++) {
123     A_for_check[i] = (double *)malloc((n + 1) * sizeof(double));
124     for (int j = 0; j < n + 1; j++) {
125         A_for_check[i][j] = A[i][j];
126     }
127 }
128 // Закрытие файла
129 fclose(fp);
130
131 // Решение
132 printf("\n=====Матрица A|b:\n");
133 for (int i = 0; i < n; i++) {
134     for (int j = 0; j < n + 1; j++) {
135         printf("%.2f ", A[i][j]);
136     }
137     printf("\n");
138 }
139
140 // Вызов функции Гаусса
141 clock_t start = clock();
142 X = gauss(A, n);
143 clock_t end = clock();
144 double time_spent = (double)(end - start) / CLOCKS_PER_SEC;
145 printf("Время выполнения: %.6f секунд\n", time_spent);
146 printf("Процессорные такты: %ld\n", (long)(end - start));
147
148 // Вывод решения
149 printf("Решение системы:\n");

```

```

150     for (int i = 0; i < n; i++) {
151         printf("X[%d] = %.6f\n", i, X[i]);
152     }
153     printf("=====\n");
154
155     double *check = (double *)malloc(n * sizeof(double));
156     check = matrix_x_vector_column(A_for_check, X, n);
157     for (int i = 0; i < n; i++) {
158         check[i] -= A_for_check[i][n]; // вычитаем b[i]
159     }
160     for (int i = 0; i < n; i++) {
161         printf("check[%d] = %.6f\n", i, check[i]);
162     }
163
164     // Освобождение памяти
165     for (int i = 0; i < n; i++) {
166         free(A[i]);
167         free(A_for_check[i]);
168     }
169     free(A);
170     free(X);
171 }
172 else { // НЕ ИСПОЛЬЗУЕТСЯ ФАЙЛ
173     int nn = 100;
174     //ввод желаемого размера матрицы через консоль
175     printf("Введите размер матрицы n: ");
176     scanf("%d", &nn);
177
178     for (n = nn; n < 100; n += 100)
179     {
180         // A --- Расширенная матрица коэффициентов (n x (n+1))
181         A = (double **)malloc(n * sizeof(double *));
182         for (int i = 0; i < n; i++) {
183             A[i] = (double *)malloc((n + 1) * sizeof(double));
184         }
185         //заполнение матрицы A и B и их объединение в PMK
186         X_target = (double *)malloc(n * sizeof(double));
187
188         int a = 1;
189         for (int i = 0; i < n; i++) {
190             X_target[i] = a;
191
192             for (int j = 0; j < n; j++) {
193                 if (i == j)
194                     A[i][j] = a;
195                 else
196                     A[i][j] = 0.1 * a;
197             }
198             a += 1;
199         }

```

```

200
201     B = matrix_x_vector_column(A, X_target, n);
202
203     for (int i = 0; i < n; i++) {
204         A[i][n] = B[i];
205     }
206
207     if (n < 10) {
208         // Решение
209         printf("\n=====Матрица A\b:\n");
210         for (int i = 0; i < n; i++) {
211             for (int j = 0; j < n + 1; j++) {
212                 printf("%.2f ", A[i][j]);
213             }
214             printf("\n");
215         }
216     }
217
218     // Вызов функции Гаусса
219     clock_t start = clock();
220     X = gauss(A, n);
221     clock_t end = clock();
222     double time_spent = (double)(end - start) / CLOCKS_PER_SEC;
223     printf("Время выполнения: %.6f секунд\n", time_spent);
224     printf("Процессорные такты: %ld\n", (long)(end - start));
225
226
227     // Вывод решения
228     printf("Решение системы:\n");
229     if (n < 10)
230         for (int i = 0; i < n; i++)
231             printf("X[%d] = %.6f\n", i, X[i]);
232     //else for (int i = 0; i < n; i++) printf("%.2f ", X[i]);
233
234     printf("=====\n");
235
236     // Освобождение памяти
237     for (int i = 0; i < n; i++) {
238         free(A[i]);
239     }
240     free(A);
241     free(X);
242     free(B);
243     free(X_target);
244 }
245 }
246
247 return 0;
248 }

```

2 Метод Ньютона решения системы нелинейных уравнений

Задача:

$$\begin{cases} f_1(x, y) = x^2 + y^2 - 4 = 0 \\ f_2(x, y) = x \cdot y - 1 = 0 \end{cases}$$

Решение:

Построив графики уравнений $f_1(x, y) = 0$ и $f_2(x, y) = 0$, можно увидеть, что они пересекаются в четырех точках, которые являются решениями данной системы нелинейных уравнений. Эти точки можно найти численно с помощью метода Ньютона, используя начальное приближение, выбранное вблизи предполагаемых пересечений.

$$\begin{aligned} f_1(x, y) &= x^2 + y^2 - 4 = 0 \\ f_2(x, y) &= x \cdot y - 1 = 0 \end{aligned}$$

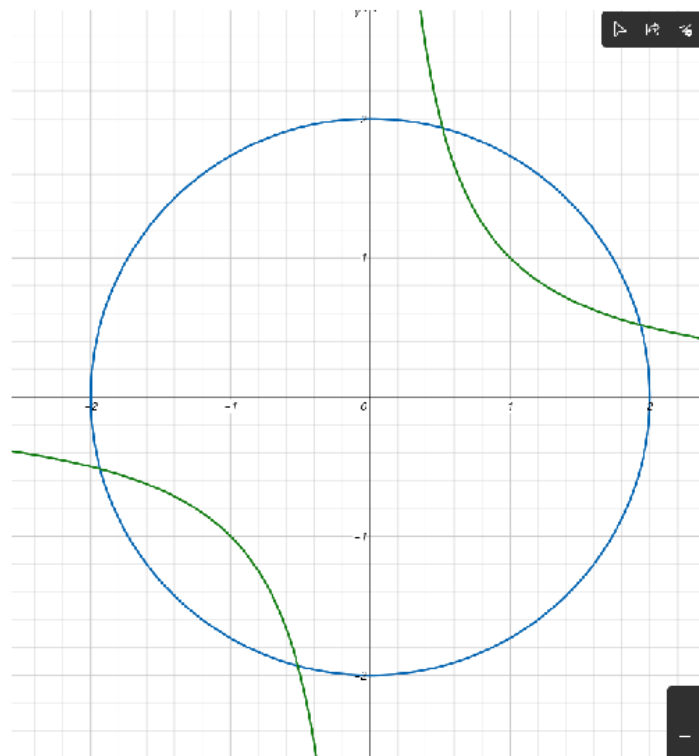


Рисунок 1 – Графики функций $f_1(x, y) = 0$ и $f_2(x, y) = 0$

В результате получаются следующие результаты:

```

> .\Newton.exe
Решение системы нелинейных уравнений методом Ньютона:
  f1(x,y) = x^2 + y^2 - 4 = 0
  f2(x,y) = x*y - 1 = 0

Введите начальное приближение (x0 y0): 3 2

Итерация 1: x = 2.30000000, y = 0.80000000, |Δ| = 1.389244e+00, |F| = 2.104875e+00
Итерация 2: x = 1.96720430, y = 0.55053763, |Δ| = 4.159140e-01, |F| = 1.918748e-01
Итерация 3: x = 1.93231527, y = 0.51809958, |Δ| = 4.763897e-02, |F| = 2.536005e-03
Итерация 4: x = 1.93185174, y = 0.51763818, |Δ| = 6.540277e-04, |F| = 4.782406e-07

Приближённое решение:
x = 1.9318517399
y = 0.5176381775

Количество итераций: 4
Норма разности между двумя последними приближениями: 6.540277e-04
Погрешность (норма невязки |F|): 4.782406e-07

```

Рисунок 2 – Начальное приближение (3, 2)

```

> .\Newton.exe
Решение системы нелинейных уравнений методом Ньютона:
  f1(x,y) = x^2 + y^2 - 4 = 0
  f2(x,y) = x*y - 1 = 0

Введите начальное приближение (x0 y0): 2 3

Итерация 1: x = 0.80000000, y = 2.30000000, |Δ| = 1.389244e+00, |F| = 2.104875e+00
Итерация 2: x = 0.55053763, y = 1.96720430, |Δ| = 4.159140e-01, |F| = 1.918748e-01
Итерация 3: x = 0.51809958, y = 1.93231527, |Δ| = 4.763897e-02, |F| = 2.536005e-03
Итерация 4: x = 0.51763818, y = 1.93185174, |Δ| = 6.540277e-04, |F| = 4.782406e-07

Приближённое решение:
x = 0.5176381775
y = 1.9318517399

Количество итераций: 4
Норма разности между двумя последними приближениями: 6.540277e-04
Погрешность (норма невязки |F|): 4.782406e-07

```

Рисунок 3 – Начальное приближение (2, 3)

```

> .\Newton.exe
Решение системы нелинейных уравнений методом Ньютона:
  f1(x,y) = x^2 + y^2 - 4 = 0
  f2(x,y) = x*y - 1 = 0

Введите начальное приближение (x0 y0): -2 -3

Итерация 1: x = -0.80000000, y = -2.30000000, |Δ| = 1.389244e+00, |F| = 2.104875e+00
Итерация 2: x = -0.55053763, y = -1.96720430, |Δ| = 4.159140e-01, |F| = 1.918748e-01
Итерация 3: x = -0.51809958, y = -1.93231527, |Δ| = 4.763897e-02, |F| = 2.536005e-03
Итерация 4: x = -0.51763818, y = -1.93185174, |Δ| = 6.540277e-04, |F| = 4.782406e-07

Приближённое решение:
x = -0.5176381775
y = -1.9318517399

Количество итераций: 4
Норма разности между двумя последними приближениями: 6.540277e-04
Погрешность (норма невязки |F|): 4.782406e-07

```

Рисунок 4 – Начальное приближение (-2, -3)


```

> .\Newton.exe
Решение системы нелинейных уравнений методом Ньютона:
  f1(x,y) = x^2 + y^2 - 4 = 0
  f2(x,y) = x*y - 1 = 0

Введите начальное приближение (x0 y0): -3 -2

Итерация 1: x = -2.30000000, y = -0.80000000, |Δ| = 1.389244e+00, |F| = 2.104875e+00
Итерация 2: x = -1.96720430, y = -0.55053763, |Δ| = 4.159140e-01, |F| = 1.918748e-01
Итерация 3: x = -1.93231527, y = -0.51809958, |Δ| = 4.763897e-02, |F| = 2.536005e-03
Итерация 4: x = -1.93185174, y = -0.51763818, |Δ| = 6.540277e-04, |F| = 4.782406e-07

Приближённое решение:
x = -1.9318517399
y = -0.5176381775

Количество итераций: 4
Норма разности между двумя последними приближениями: 6.540277e-04
Погрешность (норма невязки |F|): 4.782406e-07

```

Рисунок 5 – Начальное приближение (-3, -2)

Программная реализация на языке C:

```

1  #include <stdio.h>
2  #include <math.h>
3  #include <locale.h>
4  #include <stdlib.h>
5  #include <windows.h>
6
7  /* Определение системы нелинейных уравнений:
8     f1(x, y) = x^2 + y^2 - 4 = 0
9     f2(x, y) = x*y - 1 = 0
10 */
11
12 double f1(double x, double y) {
13     return x * x + y * y - 4.0;
14 }
15
16 double f2(double x, double y) {
17     return x * y - 1.0;
18 }
19
20 double df1dx(double x, double y) {
21     return 2.0 * x;
22 }
23
24 double df1dy(double x, double y) {
25     return 2.0 * y;
26 }
27
28 double df2dx(double x, double y) {
29     return y;
30 }
31
32 double df2dy(double x, double y) {

```

```

33     return x;
34 }
35
36 int main() {
37     SetConsoleOutputCP(CP_UTF8);
38     setlocale(LC_NUMERIC, "C");
39
40     double x0, y0, eps;
41     int max_iter;
42
43     printf("Решение системы нелинейных уравнений методом Ньютона:\n");
44     printf("  f1(x,y) = x^2 + y^2 - 4 = 0\n");
45     printf("  f2(x,y) = x*y - 1 = 0\n\n");
46
47     eps = 0.0001; // Точность по умолчанию
48     max_iter = 100; // Максимальное число итераций по умолчанию
49
50     // Ввод начального приближения
51     printf("Введите начальное приближение (x0 y0): ");
52     if (scanf("%lf %lf", &x0, &y0) != 2) {
53         printf("Ошибка: некорректный ввод начального приближения.\n");
54         return 1;
55     }
56     /*
57     // Ввод точности
58     printf("Введите точность (eps > 0): ");
59     if (scanf("%lf", &eps) != 1 || eps <= 0.0) {
60         printf("Ошибка: точность должна быть положительным числом.\n");
61         return 1;
62     }
63
64     // Ввод максимального числа итераций
65     printf("Введите максимальное число итераций: ");
66     if (scanf("%d", &max_iter) != 1 || max_iter <= 0) {
67         printf("Ошибка: max_iter должно быть целым больше нуля.\n");
68         return 1;
69     }
70     */
71     printf("\n");
72
73     double x = x0, y = y0;
74     int iter;
75     // норма разности между двумя последними приближениями
76     double norm_delta_last = 0.0;
77     // норма невязки в последней точке
78     double norm_F_last = 0.0;
79
80     for (iter = 0; iter < max_iter; ++iter) {
81         // Вычисление значений функций и матрицы Якоби
82         double f1_val = f1(x, y);

```

```

83     double f2_val = f2(x, y);
84     double J11 = df1dx(x, y);
85     double J12 = df1dy(x, y);
86     double J21 = df2dx(x, y);
87     double J22 = df2dy(x, y);
88
89     double det = J11 * J22 - J12 * J21;
90     if (fabs(det) < 1e-12) {
91         printf("Ошибка: якобиан вырожден (det = %e) на итерации %d\n",
92             det, iter);
93         break;
94     }
95
96     // Решение системы J * delta = -F
97     double dx = (-J22 * f1_val + J12 * f2_val) / det;
98     double dy = (J21 * f1_val - J11 * f2_val) / det;
99
100    double x_new = x + dx;
101    double y_new = y + dy;
102
103    // Норма разности между приближениями
104    norm_delta_last = sqrt(dx * dx + dy * dy);
105
106    // Вычисление невязки в новом приближении
107    double f1_new = f1(x_new, y_new);
108    double f2_new = f2(x_new, y_new);
109    norm_F_last = sqrt(f1_new * f1_new + f2_new * f2_new);
110
111    // Печать информации об итерации
112    printf("Итерация%2d: x = %.8lf, y = %.8lf, |delta| = %e, |F| = %e\n",
113        iter + 1, x_new, y_new, norm_delta_last, norm_F_last);
114
115    // Обновление переменных
116    x = x_new;
117    y = y_new;
118
119    // Проверка критериев остановки
120    if (norm_delta_last < eps || norm_F_last < eps) {
121        break;
122    }
123 }
124
125 // Вывод результатов
126 printf("\nПриближённое решение:\n");
127 printf("x = %.10lf\n", x);
128 printf("y = %.10lf\n", y);
129 printf("\nКоличество итераций: %d\n", iter + 1);
130 printf("Норма разности между двумя последними приближениями: %e\n",
131     norm_delta_last);
132 printf("Погрешность (норма невязки |F|): %e\n", norm_F_last);

```

```

133
134     if (iter == max_iter && norm_delta_last >= eps && norm_F_last >= eps) {
135         printf("\nПредупреждение: достигнуто максимальное число итераций.\n");
136     }
137
138     return 0;
139 }

```

3 Нахождение собственных значений собственных векторов матрицы

Задача: найти собственные значения и собственные векторы матрицы:

$$A = \begin{pmatrix} 5 & 4 & 3 & 2 & 1 \\ 4 & 5 & 4 & 3 & 2 \\ 3 & 4 & 5 & 4 & 3 \\ 2 & 3 & 4 & 5 & 4 \\ 1 & 2 & 3 & 4 & 5 \end{pmatrix}$$

Программа получает на вход квадратную матрицу, точность и максимальное число итераций. В результате выводятся собственные значения в порядке убывания, соответствующие им собственные векторы, а также проверка невязки $Av - \lambda v$ для каждого найденного собственного вектора.

Вывод программы:

Ввод матрицы:
 1 - из файла matrix.txt 2 - с клавиатуры
 Ваш выбор: 1

Введённая матрица:

Исходная матрица:

5.000000	4.000000	3.000000	2.000000	1.000000
4.000000	5.000000	4.000000	3.000000	2.000000
3.000000	4.000000	5.000000	4.000000	3.000000
2.000000	3.000000	4.000000	5.000000	4.000000
1.000000	2.000000	3.000000	4.000000	5.000000

Введите точность (eps > 0): 0.001
 Введите максимальное число итераций: 100

=== РЕЗУЛЬТАТЫ ===
 Количество итераций QR-алгоритма: 19

Собственные значения (в порядке убывания):
 lambda1 = 17.177798
 lambda2 = 5.236068
 lambda3 = 1.273771

```
lambda4 = 0.763930
lambda5 = 0.548434
```

Собственные векторы (по столбцам):

0.388330	-0.601501	0.548416	-0.372285	0.219180
0.472560	-0.371748	-0.130006	0.603048	-0.507860
0.501771	0.000000	-0.604132	-0.002085	0.619069
0.472560	0.371748	-0.129686	-0.599948	-0.511599
0.388330	0.601501	0.548218	0.371207	0.221491

=== ПРОВЕРКА $A*v - \lambda*v$ ===

Невязка для $\lambda_1 = 2.909515e-09$

Невязка для $\lambda_2 = 2.909583e-09$

Невязка для $\lambda_3 = 1.357215e-04$

Невязка для $\lambda_4 = 6.833356e-04$

Невязка для $\lambda_5 = 6.697217e-04$

Максимальная невязка: $6.833356e-04$

Программная реализация на языке C:

```
1 #include <stdio.h>
2 #include <math.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <locale.h>
6 #include <windows.h>
7
8 // ОСНОВНЫЕ ФУНКЦИИ
9
10 double** alloc_matrix(int n) {
11     double** A = (double**)malloc(n * sizeof(double*));
12     if (A == NULL) return NULL;
13     for (int i = 0; i < n; i++) {
14         A[i] = (double*)malloc(n * sizeof(double));
15         if (A[i] == NULL) {
16             for (int j = 0; j < i; j++) free(A[j]);
17             free(A);
18             return NULL;
19         }
20     }
21     return A;
22 }
23
24 void free_matrix(double** A, int n) {
25     if (A == NULL) return;
26     for (int i = 0; i < n; i++) free(A[i]);
27     free(A);
28 }
29
30 void copy_matrix(double** dest, double** src, int n) {
31     for (int i = 0; i < n; i++)
```

```

32         for (int j = 0; j < n; j++)
33             dest[i][j] = src[i][j];
34     }
35
36 void matrix_multiply(double** A, double** B,
37                     double** C, int n) {
38     double** temp = alloc_matrix(n);
39     if (temp == NULL) return;
40     for (int i = 0; i < n; i++) {
41         for (int j = 0; j < n; j++) {
42             temp[i][j] = 0.0;
43             for (int k = 0; k < n; k++)
44                 temp[i][j] += A[i][k] * B[k][j];
45         }
46     }
47     copy_matrix(C, temp, n);
48     free_matrix(temp, n);
49 }
50
51 double off_diagonal_norm(double** A, int n) {
52     double sum = 0.0;
53     for (int i = 0; i < n; i++)
54         for (int j = 0; j < n; j++)
55             if (i != j) sum += A[i][j] * A[i][j];
56     return sqrt(sum);
57 }
58
59 void print_matrix(double** A, int n, const char* title) {
60     printf("\n%s:\n", title);
61     for (int i = 0; i < n; i++) {
62         for (int j = 0; j < n; j++)
63             printf("%12.6f ", A[i][j]);
64         printf("\n");
65     }
66 }
67
68 void print_eigenvalues(double* eigenvals, int n) {
69     printf("\nСобственные значения (в порядке убывания):\n");
70     for (int i = 0; i < n; i++)
71         printf("lambda%d = %12.6f\n", i+1, eigenvals[i]);
72 }
73
74 void print_eigenvectors(double** V, int n) {
75     printf("\nСобственные векторы (по столбцам):\n");
76     for (int i = 0; i < n; i++) {
77         for (int j = 0; j < n; j++)
78             printf("%12.6f ", V[i][j]);
79         printf("\n");
80     }
81 }

```

```

82
83 // ===== QR-РАЗЛОЖЕНИЕ (ВРАЩЕНИЯ ГИВЕНСА)
84 void qr_decomposition(double** A, double** Q,
85     double** R, int n) {
86     for (int i = 0; i < n; i++)
87         for (int j = 0; j < n; j++)
88             R[i][j] = A[i][j];
89     for (int i = 0; i < n; i++)
90         for (int j = 0; j < n; j++)
91             Q[i][j] = (i == j) ? 1.0 : 0.0;
92
93     for (int j = 0; j < n-1; j++) {
94         for (int i = n-1; i > j; i--) {
95             if (fabs(R[i][j]) > 1e-14) {
96                 double a = R[i-1][j];
97                 double b = R[i][j];
98                 double r = sqrt(a*a + b*b);
99                 double c = a / r;
100                 double s = -b / r;
101                 for (int k = j; k < n; k++) {
102                     double temp = c * R[i-1][k] - s * R[i][k];
103                     R[i][k] = s * R[i-1][k] + c * R[i][k];
104                     R[i-1][k] = temp;
105                 }
106                 for (int k = 0; k < n; k++) {
107                     double temp = c * Q[k][i-1] - s * Q[k][i];
108                     Q[k][i] = s * Q[k][i-1] + c * Q[k][i];
109                     Q[k][i-1] = temp;
110                 }
111             }
112         }
113     }
114 }
115
116 // ===== QR-АЛГОРИТМ
117 int qr_algorithm(double** A, double* eigenvalues,
118     double** eigenvectors, int n, double eps,
119     int max_iter, int* iterations_used)
120 {
121     double** H = alloc_matrix(n);
122     double** Q = alloc_matrix(n);
123     double** R = alloc_matrix(n);
124     double** newH = alloc_matrix(n);
125     double** V = alloc_matrix(n);
126     if (!H || !Q || !R || !newH || !V) {
127         printf("Ошибка выделения памяти.\n");
128         return 0;
129     }
130
131     copy_matrix(H, A, n);

```

```

132     for (int i = 0; i < n; i++)
133         for (int j = 0; j < n; j++)
134             V[i][j] = (i == j) ? 1.0 : 0.0;
135
136     int iter = 0;
137     double norm = off_diagonal_norm(H, n);
138
139     while (norm > eps && iter < max_iter) {
140         qr_decomposition(H, Q, R, n);
141         matrix_multiply(R, Q, newH, n);
142         double** temp = alloc_matrix(n);
143         matrix_multiply(V, Q, temp, n);
144         copy_matrix(V, temp, n);
145         free_matrix(temp, n);
146         copy_matrix(H, newH, n);
147         norm = off_diagonal_norm(H, n);
148         iter++;
149     }
150
151     *iterations_used = iter;
152
153     if (iter >= max_iter) {
154         printf("Внимание: достигнуто максимальное число итераций (%d)\n",
155             max_iter);
156         printf("Текущая норма внедиагональных элементов: %e\n", norm);
157         free_matrix(H, n); free_matrix(Q, n); free_matrix(R, n);
158         free_matrix(newH, n); free_matrix(V, n);
159         return 0;
160     }
161
162     for (int i = 0; i < n; i++)
163         eigenvalues[i] = H[i][i];
164
165     // Сортировка по убыванию
166     for (int i = 0; i < n-1; i++) {
167         for (int j = i+1; j < n; j++) {
168             if (eigenvalues[i] < eigenvalues[j]) {
169                 double tmp = eigenvalues[i];
170                 eigenvalues[i] = eigenvalues[j];
171                 eigenvalues[j] = tmp;
172                 for (int k = 0; k < n; k++) {
173                     double tmp_vec = V[k][i];
174                     V[k][i] = V[k][j];
175                     V[k][j] = tmp_vec;
176                 }
177             }
178         }
179     }
180
181     // Нормировка векторов

```



```

182     for (int j = 0; j < n; j++) {
183         double norm_vec = 0.0;
184         for (int i = 0; i < n; i++)
185             norm_vec += V[i][j] * V[i][j];
186         norm_vec = sqrt(norm_vec);
187         if (norm_vec > 1e-12)
188             for (int i = 0; i < n; i++)
189                 V[i][j] /= norm_vec;
190     }
191
192     copy_matrix(eigenvectors, V, n);
193
194     free_matrix(H, n); free_matrix(Q, n); free_matrix(R, n);
195     free_matrix(newH, n); free_matrix(V, n);
196     return 1;
197 }
198
199 // ===== ВВОД МАТРИЦЫ
200 double** read_matrix_from_file(int* n) {
201     const char* fname = "matrix.txt";
202     FILE* fp = fopen(fname, "r");
203     if (!fp) {
204         printf("Не удалось открыть файл %s\n", fname);
205         return NULL;
206     }
207     if (fscanf(fp, "%d", n) != 1 || *n <= 0) {
208         printf("Ошибка чтения размера матрицы.\n");
209         fclose(fp);
210         return NULL;
211     }
212     double** A = alloc_matrix(*n);
213     if (!A) return NULL;
214     for (int i = 0; i < *n; i++) {
215         for (int j = 0; j < *n; j++) {
216             if (fscanf(fp, "%lf", &A[i][j]) != 1) {
217                 printf("Ошибка чтения элемента матрицы.\n");
218                 fclose(fp);
219                 free_matrix(A, *n);
220                 return NULL;
221             }
222         }
223     }
224     fclose(fp);
225     return A;
226 }
227
228 double** read_matrix_from_keyboard(int* n) {
229     printf("Введите размерность квадратной матрицы n: ");
230     scanf("%d", n);
231     if (*n <= 0) {

```

```

232     printf("Размерность должна быть положительной.\n");
233     return NULL;
234 }
235 double** A = alloc_matrix(*n);
236 if (!A) return NULL;
237 printf("Введите элементы матрицы построчно:\n");
238 for (int i = 0; i < *n; i++)
239     for (int j = 0; j < *n; j++)
240         scanf("%lf", &A[i][j]);
241 return A;
242 }
243
244 // ===== ПРОВЕРКА  $A*v - \lambda*v = 0$ 
245 void check_residuals(double** A, double* eigenvalues,
246     double** eigenvectors, int n) {
247     double max_resid = 0.0;
248     printf("\n=== ПРОВЕРКА  $A*v - \lambda*v$  ===\n");
249     for (int j = 0; j < n; j++) {
250         double* Av = (double*)malloc(n * sizeof(double));
251         double* resid = (double*)malloc(n * sizeof(double));
252         // Вычисляем  $A * v_j$ 
253         for (int i = 0; i < n; i++) {
254             Av[i] = 0.0;
255             for (int k = 0; k < n; k++)
256                 Av[i] += A[i][k] * eigenvectors[k][j];
257             resid[i] = Av[i] - eigenvalues[j] * eigenvectors[i][j];
258         }
259         double norm_resid = 0.0;
260         for (int i = 0; i < n; i++)
261             norm_resid += resid[i] * resid[i];
262         norm_resid = sqrt(norm_resid);
263         free(Av);
264         free(resid);
265         printf("Невязка для  $\lambda$ %d = %e\n", j+1, norm_resid);
266         if (norm_resid > max_resid) max_resid = norm_resid;
267     }
268     printf("Максимальная невязка: %e\n", max_resid);
269 }
270
271 // ОСНОВНАЯ ФУНКЦИЯ
272 int main() {
273     SetConsoleOutputCP(CP_UTF8);
274     setlocale(LC_NUMERIC, "C");
275     int n, choice;
276     double** A = NULL;
277
278     printf("=== QR-АЛГОРИТМ ДЛЯ СИММЕТРИЧНОЙ МАТРИЦЫ ===\n");
279     printf("Ввод матрицы:\n1 - из файла matrix.txt");
280     printf("2 - с клавиатуры\nВаш выбор: ");
281     scanf("%d", &choice);

```

```

282
283     if (choice == 1) {
284         A = read_matrix_from_file(&n);
285     } else if (choice == 2) {
286         A = read_matrix_from_keyboard(&n);
287     } else {
288         printf("Неверный выбор.\n");
289         return 1;
290     }
291
292     if (A == NULL) {
293         printf("Ошибка при чтении матрицы.\n");
294         return 1;
295     }
296
297     printf("\nВведённая матрица:\n");
298     print_matrix(A, n, "Исходная матрица");
299
300     // Проверка симметричности (предупреждение)
301     int symmetric = 1;
302     for (int i = 0; i < n && symmetric; i++)
303         for (int j = i+1; j < n; j++)
304             if (fabs(A[i][j] - A[j][i]) > 1e-8)
305                 { symmetric = 0; break; }
306     if (!symmetric) {
307         printf("\nПредупреждение: матрица не симметрична!");
308         printf(" Алгоритм может работать некорректно.\n");
309     }
310
311     double eps;
312     int max_iter;
313     printf("\nВведите точность (eps > 0): ");
314     scanf("%lf", &eps);
315     printf("Введите максимальное число итераций: ");
316     scanf("%d", &max_iter);
317
318     double* eigenvalues = (double*)malloc(n * sizeof(double));
319     double** eigenvectors = alloc_matrix(n);
320     if (eigenvalues == NULL || eigenvectors == NULL) {
321         printf("Ошибка выделения памяти.\n");
322         free_matrix(A, n);
323         return 1;
324     }
325
326     int iter_used = 0;
327     int success = qr_algorithm(A, eigenvalues, eigenvectors, n, eps, max_iter, &iter_used);
328
329     if (success) {
330         printf("\n=== РЕЗУЛЬТАТЫ ===\n");
331         printf("Количество итераций QR-алгоритма: %d\n", iter_used);

```

```
332     print_eigenvalues(eigenvalues, n);
333     print_eigenvectors(eigenvectors, n);
334
335     check_residuals(A, eigenvalues, eigenvectors, n);
336 } else {
337     printf("\nQR-алгоритм не сошёлся за %d итераций.\n", max_iter);
338 }
339
340 free_matrix(A, n);
341 free_matrix(eigenvectors, n);
342 free(eigenvalues);
343
344 return 0;
345 }
346
```